

PART 1: DERIVING BACKWARD FOR NEW FUNCTIONS

THE FORMULA FOR BACKWARD:

For function $v = f(p_1, p_2, \dots, p_k)$:

backward returns: $[\partial f / \partial p_1 \times \text{output_grad}, \partial f / \partial p_2 \times \text{output_grad}, \dots]$

- Take derivative of f with respect to each parent
- Multiply each by `output_grad` (chain rule)
- Return list with one entry per parent

COMMON FUNCTIONS — MEMORIZE THESE:

FUNCTION	$f(\text{parents})$	backward returns
<code>add(a, b)</code>	$a + b$	<code>[output_grad, output_grad]</code>
<code>sub(a, b)</code>	$a - b$	<code>[output_grad, -output_grad]</code>
<code>mul(a, b)</code>	$a \times b$	<code>[b × output_grad, a × output_grad]</code>
<code>div(a, b)</code>	a / b	<code>[output_grad/b, -a×output_grad/b²]</code>
<code>square(a)</code>	a^2	<code>[2 × a × output_grad]</code>
<code>power(a, n)</code>	a^n (n const)	<code>[n × a⁽ⁿ⁻¹⁾ × output_grad]</code>
<code>sqrt(a)</code>	$\sqrt{a}$	<code>[output_grad / (2×√a)]</code>
<code>exp(a)</code>	e^a	<code>[e^a × output_grad]</code>
<code>log(a)</code>	$\ln(a)$	<code>[output_grad / a]</code>
<code>sin(a)</code>	$\sin(a)$	<code>[cos(a) × output_grad]</code>
<code>cos(a)</code>	$\cos(a)$	<code>[-sin(a) × output_grad]</code>
<code>relu(a)</code>	$\max(0, a)$	<code>[(a > 0) × output_grad]</code>
<code>neg(a)</code>	$-a$	<code>[-output_grad]</code>
<code>max(a, b)</code>	$\max(a, b)$	<code>[(a>b)×out_grad, (b≥a)×out_grad]</code>
<code>min(a, b)</code>	$\min(a, b)$	<code>[(a<b)×out_grad, (b≤a)×out_grad]</code>

HOW TO DERIVE BACKWARD FOR ANY NEW FUNCTION:

- Step 1: Write out $f(p_1, p_2, \dots, p_k)$
 Step 2: Compute $\partial f / \partial p_i$ for each parent p_i
 Step 3: Multiply each by `output_grad`
 Step 4: Return as list `[grad_p1, grad_p2, ..., grad_pk]`

EXAMPLE: Derive backward for $f(a, b) = a^2 + 2ab + b^2$
 $\partial f / \partial a = 2a + 2b, \quad \partial f / \partial b = 2a + 2b$
 backward returns: `[(2a + 2b) × output_grad, (2a + 2b) × output_grad]`

EXAMPLE: Derive backward for $f(a, b, c) = a \times b + b \times c$
 $\partial f / \partial a = b, \quad \partial f / \partial b = a + c, \quad \partial f / \partial c = b$
 backward returns: `[b × out_grad, (a+c) × out_grad, b × out_grad]`

PART 2: CORRECT ALGORITHM — REFERENCE FOR COMPARISON

CORRECT FORWARD PASS:

```
def eval_graph(input_nodes, input_values, output_nodes):
    all_nodes = upstream_nodes(output_nodes)    # topological order
    values = {n: v for n, v in zip(input_nodes, input_values)}
    for node in all_nodes:
        if node.function is None: continue        # FORWARD through topo order
        parent_values = [values[p] for p in node.parents]
        values[node] = node.function(*parent_values)
    return values
```

CORRECT BACKWARD PASS (BACKPROPAGATION):

```
def eval_grad(input_nodes, input_values, output_node):
    all_nodes = upstream_nodes([output_node])
    values = eval_graph(input_nodes, input_values, [output_node])
    grads = {node: 0.0 for node in all_nodes}    # Initialize ALL to 0
    grads[output_node] = 1.0                    # Output grad = 1
```

```

for node in reversed(all_nodes):          # Δ REVERSE topo order!
    if node.function is None: continue    # skip input nodes
    parent_values = [values[p] for p in node.parents]
    parent_grads = backward(node.function, parent_values, grads[node])
    for p, pg in zip(node.parents, parent_grads):
        grads[p] += pg                    # Δ ACCUMULATE with +=
    return [grads[p] for p in input_nodes]

```

PART 3: ANALYZING PSEUDOCODE — IS IT RIGHT OR WRONG?

Δ THE 4 CRITICAL REQUIREMENTS FOR CORRECT BACKPROP:

1. REVERSE TOPOLOGICAL ORDER — process nodes from output → input
2. ACCUMULATE GRADIENTS WITH += — not assignment (=)
3. INITIALIZE ALL GRADIENTS TO 0 — output node gradient = 1
4. CORRECT CHAIN RULE — propagate grads[node] to PARENTS

COMMON ERRORS TO LOOK FOR:

ERROR 1: Wrong traversal direction

```

WRONG: for node in all_nodes:             # Forward order!
RIGHT: for node in reversed(all_nodes):    # Reverse order!
Why: Need children processed before parents so grads[child] is ready

```

ERROR 2: Assignment instead of accumulation

```

WRONG: grads[p] = pg                      # Overwrites!
RIGHT: grads[p] += pg                     # Accumulates!
Why: Node with 2+ children loses earlier children's contributions

```

ERROR 3: Processing wrong direction (parents vs children)

```

CORRECT: For each node, update PARENTS based on grads[node]
ALSO OK: For each node, sum from CHILDREN into grads[node]
Both work IF implemented correctly with proper accumulation

```

ERROR 4: Not initializing gradients

```

WRONG: (no initialization)
RIGHT: grads = {node: 0.0 for node in all_nodes}
       grads[output_node] = 1.0

```

ERROR 5: Processing input nodes (they have no function!)

```

Must skip: if node.function is None: continue

```

PART 4: MIDTERM-STYLE QUESTION ANALYSIS

ANALYZING VARIANT 1 (from midterm):

```

g[v_out] ← 1
for each node v in reverse topological order:
    for all parents u of node v:
        g[u] ← g[v] · ∂v/∂u          ← PROBLEM HERE!

```

ANSWER: NO — does NOT compute correct gradients.

WHY WRONG: Uses ASSIGNMENT (←) instead of ACCUMULATION (+=).

EXAMPLE where it fails:

```

      a
     / \
    b   c      (both b and c have a as parent)
     \ /
      d

```

- When processing b: $g[a] \leftarrow g[b] \times \partial b / \partial a$
- When processing c: $g[a] \leftarrow g[c] \times \partial c / \partial a$ ← OVERWRITES first!
- Lost the contribution from b!

FIX: Change ← to +=

```

g[u] += g[v] · ∂v/∂u

```

ANALYZING VARIANT 2 (from midterm):

```

For every node v: g[v] ← 0

```

```

g[v_out] ← 1
for each node u in reverse topological order:
  for all children v of node u:
    g[u] ← g[u] + g[v] · ∂v/∂u

```

ANSWER: YES — this DOES compute correct gradients.

WHY CORRECT:

- Initializes all gradients to 0 ✓
- Sets output gradient to 1 ✓
- Processes in reverse topological order ✓
- Uses accumulation (g[u] + ...) ✓
- For each node u, sums contributions from ALL its children ✓

NOTE: Requires knowing children of each node (extra preprocessing), but same asymptotic complexity as standard algorithm.

PART 5: QUICK DECISION FLOWCHART FOR PSEUDOCODE ANALYSIS

STEP 1: Check traversal order

- ☐ Is it REVERSE topological order?
- If forward order → WRONG

STEP 2: Check gradient initialization

- ☐ Are all gradients initialized to 0?
- ☐ Is output gradient set to 1?
- If missing → WRONG

STEP 3: Check accumulation vs assignment

- ☐ Does it use += or equivalent (g[x] = g[x] + ...)?
- If just = without adding previous value → WRONG

STEP 4: Check chain rule direction

- ☐ Is it multiplying by correct local derivative?
- ☐ Is gradient flowing from output toward inputs?

TEMPLATE ANSWER:

"[YES/NO], this code [does/does not] compute correct gradients.

[If NO]: The problem is [specific issue].

Example: Consider graph where node X has multiple [children/parents].
When processing [node], the gradient [is overwritten / not accumulated].

Fix: Change [specific line] from [wrong] to [correct]."

PART 6: THE CORE RESULT — KNOW THIS!

"The gradient also has an expression graph, and it's the same size."

Function evaluation: $O(n)$ where n = number of nodes

Gradient evaluation: $O(n)$ — SAME complexity!

"Whatever function you started with, you can always evaluate the gradient in basically the same amount of time as evaluating the original function."

"This is the core mathematical coincidence that makes machine learning possible. If this stopped existing, none of modern AI would work."

WHY SAME SIZE:

- Forward pass: visit each node once
- Backward pass: visit each node once (in reverse)
- Each node: $O(1)$ work (compute local derivatives)
- Total: $2 \times O(n) = O(n)$

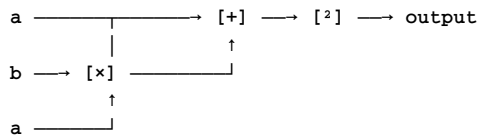
PART 7: EXPRESSION GRAPH BASICS

WHAT IS AN EXPRESSION GRAPH:

- Set of nodes, each with:
 - A function (or None for input nodes)

- An ORDERED list of parents
- No directed cycles
- Number of parents = number of function inputs

EXAMPLE: $f(a,b) = (a + a \times b)^2$



Nodes: a , b , $c=a \times b$, $d=a+c$, $e=d^2$

LIMITATIONS — WHAT CAN'T BE REPRESENTED:

- While loops (variable iterations)
- For loops with input-dependent range
- Conditional statements (unless converted to masking)

Workarounds:

- Fixed loops: unroll them
- If-else: use masking: $c = (a < b) \times f(a) + (a \geq b) \times f(b)$

PART 8: WORKED EXAMPLE — FULL BACKWARD PASS

EXAMPLE: $f(a,b) = (a + a \times b)^2$ with $a=3$, $b=1.5$

FORWARD PASS:

```

a = 3, b = 1.5
c = a * b = 4.5
d = a + c = 7.5
e = d^2 = 56.25
  
```

BACKWARD PASS:

Initialize: $\text{grads} = \{a:0, b:0, c:0, d:0, e:0\}$, $\text{grads}[e] = 1.0$

Process e (square): $\partial e / \partial d = 2d = 15$
 $\text{grads}[d] += 1.0 \times 15 = 15$

Process d (add): $\partial d / \partial a = 1$, $\partial d / \partial c = 1$
 $\text{grads}[a] += 15 \times 1 = 15$
 $\text{grads}[c] += 15 \times 1 = 15$

Process c (mul): $\partial c / \partial a = b = 1.5$, $\partial c / \partial b = a = 3$
 $\text{grads}[a] += 15 \times 1.5 = 22.5$ ← ACCUMULATES!
 $\text{grads}[b] += 15 \times 3 = 45$

FINAL: $\text{grads}[a] = 15 + 22.5 = 37.5$, $\text{grads}[b] = 45$
 Matches `jax.grad` output `[37.5, 45.0]` ✓

PART 9: KEY PHRASES FOR EXAM

For the core result:

- "Gradient evaluation takes same time as function evaluation"
- "Expression graph for gradient is same size as original"
- "This is what makes modern machine learning possible"

For backward:

- "Multiply local derivative by `output_grad` (chain rule)"
- "Return list with one gradient per parent"

For algorithm correctness:

- "Must use REVERSE topological order"
- "Must ACCUMULATE gradients with `+=`"
- "Node with multiple children receives gradient from each"

For analyzing variants:

- "Assignment (`=`) overwrites, accumulation (`+=`) sums"
- "Children must be processed before parents"